

Rust中使用**Protobuf**编译成什么样的**Rust**代码**Rust**中使用**tonic**和**Protobuf**多路复用的**grpc**更多**tonic**使用方式

- **Rust**中使用**Protobuf**
- **Protobuf**编译成什么样的**Rust**代码
- **Rust**中使用**tonic**和**Protobuf**
- 多路复用的**grpc**
- 更多**tonic**使用方式

Rust中使用Protobuf

对于**Rust**语言来说，有**quick-protobuf**、**rust-protobuf**、**prost**等第三方**crates**可编译**Protobuf**文件。

目前最流行的是**prost**，但它需要配合使用 `prost-build` 这个**crate**来进行编译转换。此外，如果**protobuf**要配合**tonic gRPC**使用，则可以替换 `prost-build` 为 `tonic-build` 来编译转换为适配**tonic**的结构。

以配合**tonic**使用为例。代码结构：

```
$ cargo new proto_usage
$ mkdir proto_usage/protos
$ touch proto_usage/protos/voting.proto
$ touch proto_usage/build.rs
$ tree proto_usage
proto_usage
├── Cargo.toml
├── build.rs
├── protos
│   └── voting.proto
└── src
    └── main.rs
```

Cargo.toml的内容：

```
[dependencies]
prost = "0.11.0"
tokio = { version = "1.21.0", features = ["macros", "rt-multi-thread"] }
tonic = "0.8.1"

[build-dependencies]
tonic-build = "0.8.0"
```

voting.proto文件内容：

```

syntax = "proto3";

package voting;

service Voting {
  rpc Vote(VotingRequest) returns (VotingResponse);
}

message VotingRequest {
  string url = 1;
  enum Vote {
    UP = 0;
    DOWN = 1;
  }

  Vote vote = 2;
}

message VotingResponse { string confirmation = 1; }

```

build.rs文件中编写在编译期间编译**proto**文件的代码：

```

use std::io::Result;

fn main() -> Result<()> {
    tonic_build::compile_protos("protos/voting.proto")?;
    Ok(())
}

```

在编译期间，**build.rs**将会编译 `voting.proto` 文件并转换为 `voting.rs` 文件，得到**voting.rs**文件后，就可以在需要的地方导入该文件，导入该文件后就会拥有该文件中定义的数据结构等内容。至于**voting.proto**文件编译转换后得到的**voting.rs**文件中到底是什么内容，见后文分析。

例如，在**main.rs**中导入该文件。**src/main.rs**文件的内容(此处示例仅导入但并没有使用)：

```

pub mod voting {
    tonic::include_proto!("voting");
}

fn main() {}

```

默认情况下，**build.rs**编译得到的中间文件保存在 `OUT_DIR` 环境变量指定的目录中，如果没有明确设置 `OUT_DIR` 环境变量，则默认为**cargo**的构建目录。例如

`<target_dir>\debug\build\proto_usage-4e6f1dc6a899518f\out\voting.rs`。如果没有修改过 `OUT_DIR` 环境变量，则可以通过 `tonic::include_proto!("voting")` 宏直接导入**build.rs**编译后得到的 `voting.rs` 文件。

如果修改过 `OUT_DIR` 环境变量的值，或者 `tonic_build` 编译时明确指定了输出路径(稍后给代码)，则不能使用 `tonic::include_proto!` 宏来导入 `voting.rs`。此时应该用**Rust**自身提供的宏 `include!` 来明确指定导入的路径。参考官方手册说明

https://docs.rs/tonic/latest/tonic/macro.include_proto.html。

例如，假设**build.rs**的输出路径为**proto_usage/protos**目录：

```
pub mod voting {
    // tonic::include_proto!("voting");
    include!("../protos/voting.rs");
}
```

tonic_build 提供了方法来修改**proto**编译后 **.rs** 文件的保存路径(注意, 修改路径之后需使用 **include!** 宏来导入 **.rs** 文件):

```
use std::io::Result;

fn main() -> Result<()> {
    //tonic_build::compile_protos("protos/voting.proto"?;
    tonic_build::configure()
        .build_server(true) // 是否编译生成用于服务端的代码
        .build_client(true) // 是否编译生成用于客户端的代码
        .out_dir("protos") // 输出的路径, 此处指定为项目根目录下的protos目录
        // 指定要编译的proto文件路径列表, 第二个参数是提供protobuf的扩展路径,
        // 因为protobuf官方提供了一些扩展功能, 自己也可能会写一些扩展功能,
        // 如存在, 则指定扩展文件路径, 如果没有, 则指定为proto文件所在目录即可
        .compile(&["protos/voting.proto"], &["protos"])?;
    Ok(())
}
```

Protobuf编译成什么样的Rust代码

对于前文的**voting.proto**文件:

```
syntax = "proto3";

package voting;

service Voting {
    rpc Vote(VotingRequest) returns (VotingResponse);
}

message VotingRequest {
    string url = 1;
    enum Vote {
        UP = 0;
        DOWN = 1;
    }

    Vote vote = 2;
}

message VotingResponse { string confirmation = 1; }
```

将其编译后得到**voting.rs**文件, 该文件内容并不复杂, 可以直接打开看。

更直观的方式在导入它之后,

```
pub mod voting {
    include!("../protos/voting.rs");
}
```

通过文档来了解它包含了什么：

```
$ cargo doc --open
```

可以看到，在**voting**模块下，拥有如下内容：

```
voting::{
    // 模块部分以及模块中的结构
    // 为客户端生成的代码
    voting_client::{self, VotingClient},
    // 为服务端生成的代码
    // Voting是Trait
    voting_server::{self, VotingServer, Voting},
    // 嵌套在VotingRequest中的Vote类型
    voting_request::{self, Vote},

    // Structs部分
    VotingRequest,
    VotingResponse
}
```

voting_request、**VotingRequest**和**VotingResponse**结构：

```
// voting_request中包含了VotingRequest内嵌的enum类型
pub mod voting_request {
    #[derive(Clone, Copy, Debug, ...)]
    #[repr(i32)]
    pub enum Vote {
        Up = 0,
        Down = 1,
    }
}

// 请求类型
#[derive(Clone, PartialEq, ::prost::Message)]
pub struct VotingRequest {
    pub url: String,
    pub vote: i32,
}

// 内嵌的enum Vote类型和VotingRequest vote字段之间的转换设置
impl VotingRequest {
    pub fn vote(&self) -> Vote
    pub fn set_vote(&mut self, value: Vote)
}

// 响应类型
#[derive(Clone, PartialEq, ::prost::Message)]
pub struct VotingResponse {
    pub confirmation: String,
}
```

这些字段都是**pub**公开的，因此可以直接构建这些类型或直接进行字段赋值。

为客户端生成的 `voting_client::VotingClient` 类型，有几个方法需要注意：

```

pub fn new(inner: T) -> Self

// 连接服务端并返回VotingClient
pub async fn connect<D>(dst: D) -> Result<Self, Error>

// 在proto文件中通过rpc定义的方法vote，客户端和服务端都有
// 在客户端，它会向服务端发送请求
pub async fn vote(
    &mut self,
    request: impl IntoRequest<VotingRequest>
) -> Result<Response<VotingResponse>, Status>

```

另外注意，**VotingClient**实现了**Clone**、**Sync**以及**Send**，因此可以跨线程使用。

例如：

```

#[tokio::main]
async fn main() -> Result<(), Box<dyn std::error::Error>> {
    let mut client = VotingClient::connect("http://127.0.0.1:8080").await?;
    loop {
        .....
        let url = ...;
        let vote = ...;
        // 构建请求
        let request = tonic::Request::new(VotingRequest {url, vote});
        // 发送请求并等待响应
        let response = client.vote(request).await?;
        println!("Got: '{}{}' from service", response.get_ref().confirmation);
    }
    Ok(())
}

```

为服务端生成的 `voting_server::VotingServer`：

```

impl<T: Voting> VotingServer<T> {
    pub fn new(inner: T) -> Self
    pub fn from_arc(inner: Arc<T>) -> Self
}

```

其中 `T: Voting` 中的**Trait Voting**要求实现 `vote()` 方法，该方法用于定义服务端接收到客户端请求时要如何处理的逻辑。

例如：

```

#[derive(Debug, Default)]
pub struct VotingService {}

#[tonic::async_trait]
impl Voting for VotingService {
    async fn vote(
        &self,
        request: Request<VotingRequest>,
    ) -> Result<Response<VotingResponse>, Status> {
        let r: &VotingRequest = request.get_ref();
        match r.vote {
            0 => Ok(Response::new(voting::VotingResponse {
                confirmation: { format!("upvoted for {}", r.url) },
            })),
            1 => Ok(Response::new(voting::VotingResponse {
                confirmation: { format!("downvoted for {}", r.url) },
            })),
            _ => Err(Status::new(
                tonic::Code::OutOfRange,
                "Invalid vote provided",
            )),
        }
    }
}

#[tokio::main]
async fn main() {
    let voting_service = VotingService::default();
    let server = VotingServer::new(voting_service);
}

```

Rust中使用tonic和Protobuf

将上面的代码综合起来，实现一个通过**gRPC**通信的客户端和服务端。

```

$ cargo new tonic_grpc
$ mkdir tonic_grpc/protos
$ touch tonic_grpc/protos/voting.proto
$ touch tonic_grpc/build.rs
$ touch tonic_grpc/src/{client.rs, server.rs}

$ tree tonic_grpc
tonic_grpc/
├── Cargo.toml
├── build.rs
├── protos
│   └── voting.proto
└── src
    ├── client.rs
    ├── main.rs
    └── server.rs

```

Cargo.toml内容：

```

[dependencies]
prost = "0.11.0"
tokio = { version = "1.21.0", features = ["macros", "rt-multi-thread"] }
tonic = "0.8.1"

[build-dependencies]
tonic-build = "0.8.0"

[[bin]]
name = "server"
path = "src/server.rs"

[[bin]]
name = "client"
path = "src/client.rs"

```

protos/voting.proto内容:

```

syntax = "proto3";

package voting;

service Voting {
  rpc Vote(VotingRequest) returns (VotingResponse);
}

message VotingRequest {
  string url = 1;
  enum Vote {
    UP = 0;
    DOWN = 1;
  }

  Vote vote = 2;
}

message VotingResponse { string confirmation = 1; }

```

build.rs内容:

```

use std::io::Result;

fn main() -> Result<()> {
  // tonic_build::compile_protos("protos/voting.proto");
  tonic_build::configure()
    .build_server(true)
    .build_client(true)
    .out_dir("protos")
    .compile(&["protos/voting.proto"], &["protos"]);
  Ok(())
}

```

src/server.rs内容:

```

use tonic::{transport::Server, Request, Response, Status};
use voting::{
    voting_server::{Voting, VotingServer},
    VotingRequest, VotingResponse,
};

pub mod voting {
    // tonic::include_proto!("voting");
    include!("../protos/voting.rs");
}

#[derive(Debug, Default)]
pub struct VotingService {}

#[tonic::async_trait]
impl Voting for VotingService {
    async fn vote(
        &self,
        request: Request<VotingRequest>,
    ) -> Result<Response<VotingResponse>, Status> {
        let r: &VotingRequest = request.get_ref();
        match r.vote {
            0 => Ok(Response::new(voting::VotingResponse {
                confirmation: { format!("upvoted for {}", r.url) },
            })),
            1 => Ok(Response::new(voting::VotingResponse {
                confirmation: { format!("downvoted for {}", r.url) },
            })),
            _ => Err(Status::new(
                tonic::Code::OutOfRange,
                "Invalid vote provided",
            )),
        }
    }
}

#[tokio::main]
async fn main() -> Result<(), Box<dyn std::error::Error>> {
    let address = "[::1]:8080".parse().unwrap();
    let voting_service = VotingService::default();

    Server::builder()
        .add_service(VotingServer::new(voting_service))
        .serve(address)
        .await?;
    Ok(())
}

```

src/client.rs内容:


```

use std::io::stdin;

use voting::{voting_client::VotingClient, VotingRequest};

use crate::voting::voting_request;

pub mod voting {
    // tonic::include_proto!("voting");
    include!("../protos/voting.rs");
}

#[tokio::main]
async fn main() -> Result<(), Box<dyn std::error::Error>> {
    let mut client = VotingClient::connect("http://[::1]:8080").await?;
    let url = "http://helloworld.com/post1";
    loop {
        let mut vote: String = String::new();
        println!("Voting for <{}>, (d)own or (u)p: ", url);
        stdin().read_line(&mut vote).unwrap();
        let vote_res = match vote.trim().to_lowercase().chars().next().unwrap()
        {
            'u' => voting_request::Vote::Up,
            'd' => voting_request::Vote::Down,
            _ => break,
        };
        // here comes the service invocation
        let request = tonic::Request::new(VotingRequest {
            url: String::from(url),
            vote: vote_res.into(),
        });
        let response = client.vote(request).await?;
        println!("Got: '{}' from service", response.get_ref().confirmation);
    }
    Ok(())
}

```

运行：

```

# 终端1, 运行server端:
$ cargo run --bin server

# 终端2, 运行client端:
$ cargo run --bin client
Voting for <http://helloworld.com/post1>, (d)own or (u)p:
u
Got: 'upvoted for http://helloworld.com/post1' from service
Voting for <http://helloworld.com/post1>, (d)own or (u)p:
d
Got: 'downvoted for http://helloworld.com/post1' from service
Voting for <http://helloworld.com/post1>, (d)own or (u)p:
d

```

多路复用的gRPC

如果某客户端和某服务端之间有多个gRPC通信需求，则可以复用连接。

例如，有两个proto文件：**protos/voting.proto**和**protos/hello.proto**。内容分别如下：

```
// voting.proto的内容
syntax = "proto3";
package voting;

service Voting {
  rpc Vote(VotingRequest) returns (VotingResponse);
}

message VotingRequest {
  string url = 1;
  enum Vote {
    UP = 0;
    DOWN = 1;
  }
  Vote vote = 2;
}

message VotingResponse { string confirmation = 1; }

// hello.proto的内容
syntax = "proto3";
package hello;

service Greeter {
  rpc SayHello(HelloReq) returns(HelloResp);
}

message HelloReq {
  string content = 1;
}

message HelloResp {
  string content = 1;
}
```

build.rs文件内容:

```
use std::io::Result;

fn main() -> Result<()> {
  tonic_build::compile_protos("protos/voting.proto"?;
    tonic_build::compile_protos("protos/hello.proto"?;
    Ok(())
}
```

src/client.rs文件内容:

```

use greet::{greeter_client::GreeterClient, HelloReq};
use tonic::transport::{Channel, Endpoint};
use voting::{voting_client::VotingClient, VotingRequest};
use crate::voting::voting_request;

pub mod voting { tonic::include_proto!("voting"); }
pub mod greet { tonic::include_proto!("hello"); }

type ThisErr = Box<dyn std::error::Error>;

async fn voting(client: &mut VotingClient<Channel>) -> Result<(), ThisErr> {
    let url = "http://helloworld.com/post1";
    let mut n = 0;
    loop {
        let vote_res = if n % 2 == 0 {
            voting_request::Vote::Up
        } else {
            voting_request::Vote::Down
        };

        let request = tonic::Request::new(VotingRequest {
            url: String::from(url),
            vote: vote_res.into(),
        });
        let response = client.vote(request).await?;
        println!("voting {}, Got: '{}'", n, response.get_ref().confirmation);
        n += 1;
        tokio::time::sleep(tokio::time::Duration::from_secs(1)).await;
    }
}

async fn greet(client: &mut GreeterClient<Channel>) -> Result<(), ThisErr> {
    let mut n = 0;
    loop {
        let hello_content = format!("hello {}", n);
        let req = tonic::Request::new(HelloReq {
            content: hello_content,
        });
        let resp = client.say_hello(req).await?;
        println!("greet {}, Got: '{}'", n, resp.get_ref().content);
        n += 1;
        tokio::time::sleep(tokio::time::Duration::from_secs(1)).await;
    }
}

#[tokio::main]
async fn main() -> Result<(), ThisErr> {
    // 构建一个transport::channel::Channel
    let channel = Endpoint::from_static("http://[::1]:8080").connect().await?;

    // 从同一个Channel构建两个客户端
    let voting_client = VotingClient::new(channel.clone());
    let greet_client = GreeterClient::new(channel);

    // 负责投票服务
    let task_voting = tokio::spawn(async move {
        let mut c = voting_client.clone();
        if let Err(e) = voting(&mut c).await {
            println!("voting error: {}", e);
        }
    });

    // 负责say hello的服务
    let task_greet = tokio::spawn(async move {

```

```
        let mut c = greet_client.clone();
        if let Err(e) = greet(&mut c).await {
            println!("greet error: {}", e);
        }
    });

    tokio::try_join!(task_greet, task_voting)?;
    Ok(())
}
```

src/server.rs文件内容:

```

use greet::{
    greeter_server::{Greeter, GreeterServer},
    HelloReq, HelloResp,
};
use tonic::{transport::Server, Request, Response, Status};
use voting::{
    voting_server::{Voting, VotingServer},
    VotingRequest, VotingResponse,
};

pub mod voting { tonic::include_proto!("voting"); }
pub mod greet { tonic::include_proto!("hello"); }

#[derive(Debug, Default)]
pub struct VotingService {}
#[tonic::async_trait]
impl Voting for VotingService {
    async fn vote(
        &self,
        request: Request<VotingRequest>,
    ) -> Result<Response<VotingResponse>, Status> {
        let r: &VotingRequest = request.get_ref();
        match r.vote {
            0 => Ok(Response::new(voting::VotingResponse {
                confirmation: { format!("upvoted for {}", r.url) },
            })),
            1 => Ok(Response::new(voting::VotingResponse {
                confirmation: { format!("downvoted for {}", r.url) },
            })),
            _ => Err(Status::new(
                tonic::Code::OutOfRange,
                "Invalid vote provided",
            )),
        }
    }
}

#[derive(Debug)]
pub struct GreetService;
#[tonic::async_trait]
impl Greeter for GreetService {
    async fn say_hello(&self, request: Request<HelloReq>) ->
    Result<Response<HelloResp>, Status> {
        let hello_str = request.into_inner().content;
        println!("greet from client: {}", hello_str);
        Ok(Response::new(HelloResp { content: hello_str }))
    }
}

#[tokio::main]
async fn main() -> Result<(), Box<dyn std::error::Error>> {
    let address = "[::1]:8080".parse().unwrap();
    let voting_service = VotingService::default();

    Server::builder()
        .add_service(VotingServer::new(voting_service))
        .add_service(GreeterServer::new(GreetService))
        .serve(address)
        .await?;
    Ok(())
}

```

更多**tonic**使用方式

tonic官方给的示例，非常详细：

<https://github.com/hyperium/tonic/tree/master/examples>。例如流式(**Stream**)的**grpc**、负载均衡、带**tls**证书验证等。

如果要编写流式**grpc**，建议看一遍

<https://github.com/hyperium/tonic/blob/master/examples/routeguide-tutorial.md>。